

Custom CPQ on Salesforce

Business Briefing

DOCUMENT CONTROL

DOCUMENT ID	LP-REVTECH-CPQ-BB-001
VERSION	1.0
STATUS	For discussion
AUTHOR	Bruno Romano
DATE	24 September 2026
AUDIENCE	Operations and Business leadership
COMPANION	Solution Design LP-REVTECH-CPQ-SD-001 (the technical specification)
PURPOSE	How requirements were established, what the system protects commercially, and a recommendation on mid-term plan changes

ABSTRACT

The technical design for the custom CPQ is built, deployed and tested. This briefing covers the part that matters to the business rather than to engineering: what commercial problem the system solves, how the requirements behind it were established and where they are still open, and a recommendation on the question of what happens when a customer wants to change plan part-way through an annual contract.

That last question is answered in section 3 with a specific recommendation, the reasoning behind it, and the commercial cost of the obvious alternative.

CONTENTS

1 What this protects, in business terms	3
2 How the requirements were established	3
2.1 Assumptions are recorded, not absorbed	3
2.2 Open questions are escalated, not guessed	4
2.3 The design changed because it was actually built	4
3 Mid-term plan changes	4
3.1 The recommendation	5
3.2 Upgrades and downgrades are not symmetrical	5
3.3 Co-termination: one renewal date per customer	5
3.4 What cancel-and-rebook would cost us	5
3.5 The model already supports this	6
3.6 The rules I would implement	6
3.7 What I would need from you before building it	7
4 Where I would take this next	7
5 Questions I would like to ask you	7

1 What this protects, in business terms

The system does one commercial job: **it makes the price floor real.**

Today a rep can discount to any number they can justify. The floor exists in a pricing deck and in people's memory. In the new model the floor lives in Salesforce, the platform cannot route around it, and every attempt to go below it is refused and recorded against the rep who tried.

Three consequences follow, and they are the business case:

Margin stops leaking silently. A deal below floor cannot be committed. Not by a rep in a hurry, not by a bug in the configurator, not by a script.

Discounting becomes measurable for the first time. Every refusal is logged with the rep, the product, the price asked and the floor. "How much margin do we give away, and who gives it away" changes from an opinion into a report.

Pricing gets an evidence base. If most deals are closing at exactly the floor, the floor is wrong, or the list price is. Today that pattern is invisible. From now on it is a quarterly number.

TABLE — WHAT CHANGES OPERATIONALLY

	TODAY	WITH THE CPQ
Price floor	A number in a deck	Enforced by the system; cannot be bypassed
Below-floor deal	Discovered at invoicing, or never	Refused at the point of sale
Who discounted, and how much	Reconstructed by hand	A standing report
What a won deal entitles the customer to	Derived by a person, later	Produced automatically the moment the deal is won
Changing a price or floor	A developer and a release	A RevOps edit, same day

That last row is deliberate. Prices, floors, contract terms and which features a plan includes are all configuration, not code. **Commercial policy changes should never wait for an engineering release.**

2 How the requirements were established

The brief named three plans and three add-ons. It did not give prices for the add-ons, floors for anything, or rules for what happens when a customer already has a feature they are being sold again. Those gaps had to be closed to build anything at all.

My approach was to close them explicitly rather than quietly.

2.1 Assumptions are recorded, not absorbed

Eight assumptions are written down in the solution design, each with a note on what it costs if it turns out to be wrong. The first one says plainly that **the add-on prices and floors are invented** — Design Refresh at \$750 with a \$600 floor,

Competitor Analysis at \$150 with a \$120 floor, Contacts (500) at \$100 with an \$80 floor — and that they are placeholders pending your confirmation.

They are held as data, so confirming or correcting them is a ten-minute change with no release.

2.2 Open questions are escalated, not guessed

Five questions were left open because they are commercial decisions that belong to this room, not to me. Two are worth raising today:

Can any rep waive the setup fee to zero? Right now, yes. This is the discount lever most likely to be abused *precisely because* the plan floor is now enforced. Pressure moves to the least-controlled lever. Enforcement relocates a leak; it does not remove it.

Is there a ceiling as well as a floor? Uplift above list price is currently unbounded. If that should be capped, it is one rule in the same place as the floor.

REC-1 · RECOMMENDATION — CLOSE THE SETUP FEE QUESTION FIRST

Of the five open questions, the freely waivable setup fee is the one with immediate revenue exposure. A floor on the plan with no floor on the setup fee gives a rep an uncontrolled discount lever on day one of go-live.

The fix is small — either a floor on the fee, or an approval step above a threshold — but it should be decided before the system is enforcing anything, not after the first quarter's numbers come in.

2.3 The design changed because it was actually built

The specification was written first and the system built second. Six things that were entirely reasonable on paper turned out to be wrong in practice, and they are recorded in the design rather than edited out of it.

One is commercially relevant to this conversation: the original plan was to run the integration on a cheaper API-only Salesforce licence. It cannot work — that licence cannot read the objects this system has to write. **The least-privilege argument survived; the cost saving did not.** It is flagged as an open question rather than buried, because it is a licensing decision, not an engineering one.

3 Mid-term plan changes

The question. A customer on Brand at \$595/month, four months into a twelve-month term, wants to move to All In at \$2,495/month. Do we cancel the subscription and create a new one, or change the existing one?

3.1 The recommendation

REC-2 · RECOMMENDATION — AMEND, NEVER CANCEL AND REBOOK

The original subscription stays alive and is amended. A change of plan is a **modification of an existing contract**, not the end of one contract and the start of another, and the system should record it the way the business actually experiences it.

Cancelling and recreating produces the right invoice and destroys everything else: the customer's history, the renewal date, and the reliability of every retention metric we report.

This is also the industry-standard treatment. Salesforce CPQ, Zuora and Chargebee all implement plan changes as amendments against a surviving contract, and for the same reasons.

3.2 Upgrades and downgrades are not symmetrical

This is the part most often got wrong. They are different commercial events and deserve different rules.

Upgrades take effect immediately. The customer is asking to pay us more. Never put friction, or a waiting period, in front of expansion revenue. The higher entitlements apply the same day and we charge the prorated difference for the remaining term. In our example: $\$1,900/\text{month} \times 8 \text{ remaining months} = \$15,200 \text{ of additional contract value}$, recognised from the date of change.

Downgrades take effect at renewal. The customer committed to twelve months at a price, and that commitment is the entire point of an annual contract. If a mid-term downgrade is freely available then we do not have annual contracts — we have monthly contracts with extra paperwork, and our ARR figure is fiction.

This asymmetry is what the market does. Of the leading SaaS companies surveyed, the large majority prorate upgrades immediately, while a majority defer downgrades to the end of the period.

The exception that must exist: the downgrade that prevents a cancellation. If the real alternative is churn, a downgrade is a save, and refusing it to defend a policy is how a \$595 customer becomes a \$0 customer. That path should stay open — but as an approved exception with a named owner, not a self-serve button.

3.3 Co-termination: one renewal date per customer

Whatever changes mid-term, **the contract end date does not move**. The upgraded plan ends on the original anniversary.

This sounds like a technical detail and is not. Without it, a customer who upgrades in month four and adds an add-on in month seven has three different renewal dates. Renewal forecasting stops working, the CS team cannot tell when to start a renewal conversation, and the customer receives invoices they cannot reconcile.

3.4 What cancel-and-rebook would cost us

TABLE — CONSEQUENCES OF TREATING A PLAN CHANGE AS A CANCELLATION PLUS A NEW SALE

WHAT BREAKS	WHAT IT DOES TO THE BUSINESS
Churn reporting	An upgrade is recorded as one cancellation plus one new sale. Gross churn looks worse than it is; new business looks better than it is. Both numbers become unusable.

WHAT BREAKS	WHAT IT DOES TO THE BUSINESS
Net revenue retention	NRR requires following the same customer's revenue over time. Severing the customer record breaks the thread — and NRR is the single metric investors and the board look at hardest.
Renewal date	Resets to twelve months from the change. We would be handing back four months of already-committed term, for free, every time someone upgrades.
Contract history	"What was this customer entitled to in March?" becomes unanswerable. That question surfaces in billing disputes and in any audit.
Service continuity	Between the cancellation and the new subscription the customer is entitled to nothing. Feature gating could lock a paying customer out of their own website.

The invoice comes out right. Everything we manage the business with comes out wrong.

3.5 The model already supports this

This was anticipated in the design, and the groundwork is already deployed:

- Entitlements carry an **active window** and are marked *superseded* rather than overwritten, so what a customer had in March survives the change.
- Subscriptions already carry their term and end date, so co-terming is a calculation, not a new concept.
- Every committed line remembers which catalog item it came from, so an amendment knows exactly what it is amending.

An amendment therefore becomes a new subscription generation that supersedes the previous one, with the entitlement history intact. **This is an extension of what exists, not a redesign.** I would estimate it as a phase of work rather than a project.

3.6 The rules I would implement

TABLE — PROPOSED TREATMENT BY SCENARIO

SCENARIO	TAKES EFFECT	CHARGING	CONTRACT END DATE	ENTITLEMENTS
Upgrade	Immediately	Prorated difference for remaining term	Unchanged	Increase same day; prior set superseded
Add-on added	Immediately	Prorated for remaining term	Unchanged (co-termed)	Increase same day
Downgrade	Next renewal	New rate from renewal	Unchanged	Scheduled; applied at renewal
Downgrade to prevent churn	Immediately, with approval	Prorated credit	Unchanged	Reduce same day; prior set superseded
Cancellation	Per contract terms	Per contract terms	Closed	Active window closed, record retained

3.7 What I would need from you before building it

These are commercial decisions, and I would rather ask than assume:

1. Is there a **floor below which a customer cannot downgrade** — can an All In customer drop to Launch, or only to Brand?
 2. On an upgrade, do we **invoice the difference immediately** or add it to the next scheduled invoice? This is a cash-flow decision, not a technical one.
 3. **Who approves** a mid-term downgrade — the AE, a sales manager, or Customer Success?
 4. Does a mid-term upgrade trigger a **new contract document** in PandaDoc, or an amendment addendum to the existing one?
 5. Is there an **uplift at renewal**, and does a mid-term upgrade reset the clock on it?
-

4 Where I would take this next

Three opportunities that the system creates, roughly in the order I would pursue them.

Approval routing for sub-floor deals. The most likely first request once reps start hitting the floor, and the design already anticipates it. Rather than a hard refusal, a below-floor deal could route for approval — the deal survives, the discount becomes a decision someone owns, and the exception rate becomes a number we can watch.

Expansion triggers from entitlement data. Once we know precisely what every customer is entitled to, we can compare it to what they actually use. A customer at 95% of their contact limit is an upsell conversation with evidence behind it, generated automatically rather than found by luck. This is the cheapest NRR improvement available here, and it is mostly reporting on data the system already produces.

Pricing informed by refusals. After two quarters of logged below-floor attempts we would know which products reps cannot sell at list, by how much, and to which customer segments. That is the most honest input a pricing review can have, and it is a byproduct of enforcement rather than a separate exercise.

5 Questions I would like to ask you

1. What proportion of deals currently close below the intended floor? If nobody knows, that is itself the answer — and the first thing this system tells you.
2. When a customer upgrades today, what actually happens operationally, and where does it break?
3. Is renewal date fragmentation already a problem for the CS team?
4. Which is the bigger commercial risk right now — discount leakage on new business, or churn and downgrade on the existing base? They point to different priorities for the next phase.
5. Who owns the catalog day to day? The system is built so RevOps changes prices without engineering, and that only pays off if a named person owns it.